Project Caravela

Technical Approach & Findings

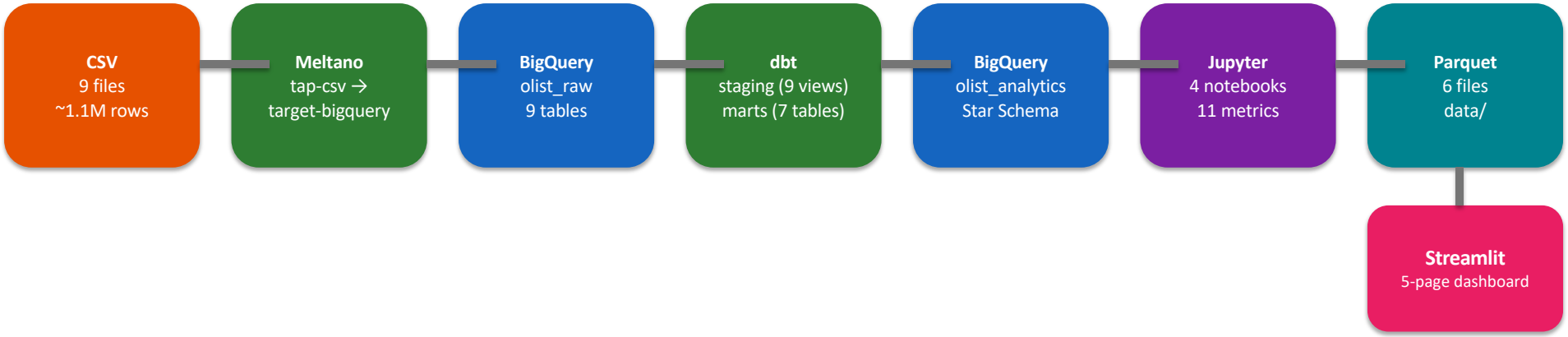
Brazilian E-Commerce Analytics Pipeline | Olist Dataset | ~100k orders, 2016–2018

5-Agent AI Architecture + Human Orchestrator

DSAI Module 2 Assignment • March 2026

Pipeline Architecture

End-to-end flow across 6 layers — modular, each independently replaceable



- **AUTOMATED — Dagster-orchestrated (daily 09:00 SGT + manual trigger)**
- **MANUAL — Notebooks, Parquet export, Streamlit dashboard**

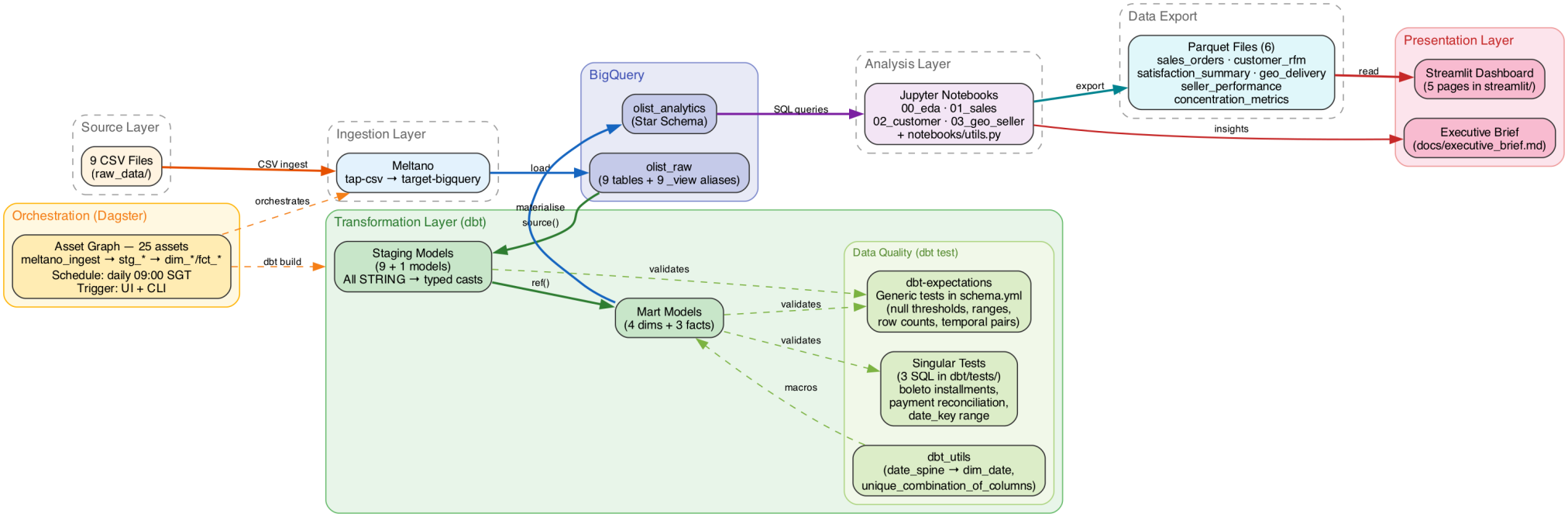
All credentials from single .env file: GOOGLE_APPLICATION_CREDENTIALS, GCP_PROJECT_ID, BIGQUERY_RAW_DATASET, BIGQUERY_ANALYTICS_DATASET

Parquet files decouple dashboard from BigQuery — Streamlit requires no GCP credentials

dbt build interleaves models + tests — failing staging test blocks dependent marts

Pipeline Architecture

End-to-end flow across 6 layers — modular, each independently replaceable



Tool Selection Rationale

Each tool chosen for a specific architectural property — decoupling is the design principle

Layer	Selected	Key Rationale	Alternatives Rejected
Ingestion	Meltano (tap-csv)	Declarative YAML; Singer protocol separates extract from load; WRITE_TRUNCATE for full refresh	Custom Python scripts bq load CLI
Transformation	dbt 1.11 on BigQuery	SQL-first ELT with ref() lineage; staging views + mart tables; all type casting centralised in staging	pandas transforms BQ stored procedures
Data Quality	dbt-expectations + singular SQL	Integrated in DAG — failing test blocks downstream marts; 76 tests, 0 failures	Great Expectations Soda
Orchestration	Dagster 1.12 + dagster-dbt	Asset-centric model; 1 asset per dbt model auto-generated; daily 09:00 SGT schedule	Airflow Prefect cron
Analysis	Jupyter + google-cloud-bigquery	Self-contained notebooks; Parquet export decouples dashboard from BigQuery	Python scripts dbt metrics
Dashboard	Streamlit 1.55	Python-native; reads Parquet (no GCP credentials); 4 global filters	Looker, Metabase Dash (Plotly)

Star Schema Design

Fact Tables

fct_sales — order-item grain (~112k rows)

Composite PK: order_id + order_item_id

4 FKs → dim_customers, dim_products, dim_sellers, dim_date

total_sale_amount = price + freight_value

3-source CTE: items → orders → customers

fct_reviews — review grain (~97k rows, post-dedup)

PK: review_id (789 source duplicates removed)

FK: order_id → stg_orders (NOT fct_sales)

⚠️ 756 itemless orders have reviews but no sales rows

fct_payments — payment-method grain

Compound PK: order_id + payment_sequential

Nullable date_key via LEFT JOIN to stg_orders

Dimension Tables

dim_customers — PK: customer_unique_id

Deduped via ROW_NUMBER; geolocation lat/lng (nullable)

dim_products — PK: product_id

COALESCE(english → portuguese → 'uncategorized')

7 nullable numeric columns via SAFE_CAST

dim_sellers — PK: seller_id

Geolocation enrichment by zip code prefix

dim_date — PK: date_key (DATE type)

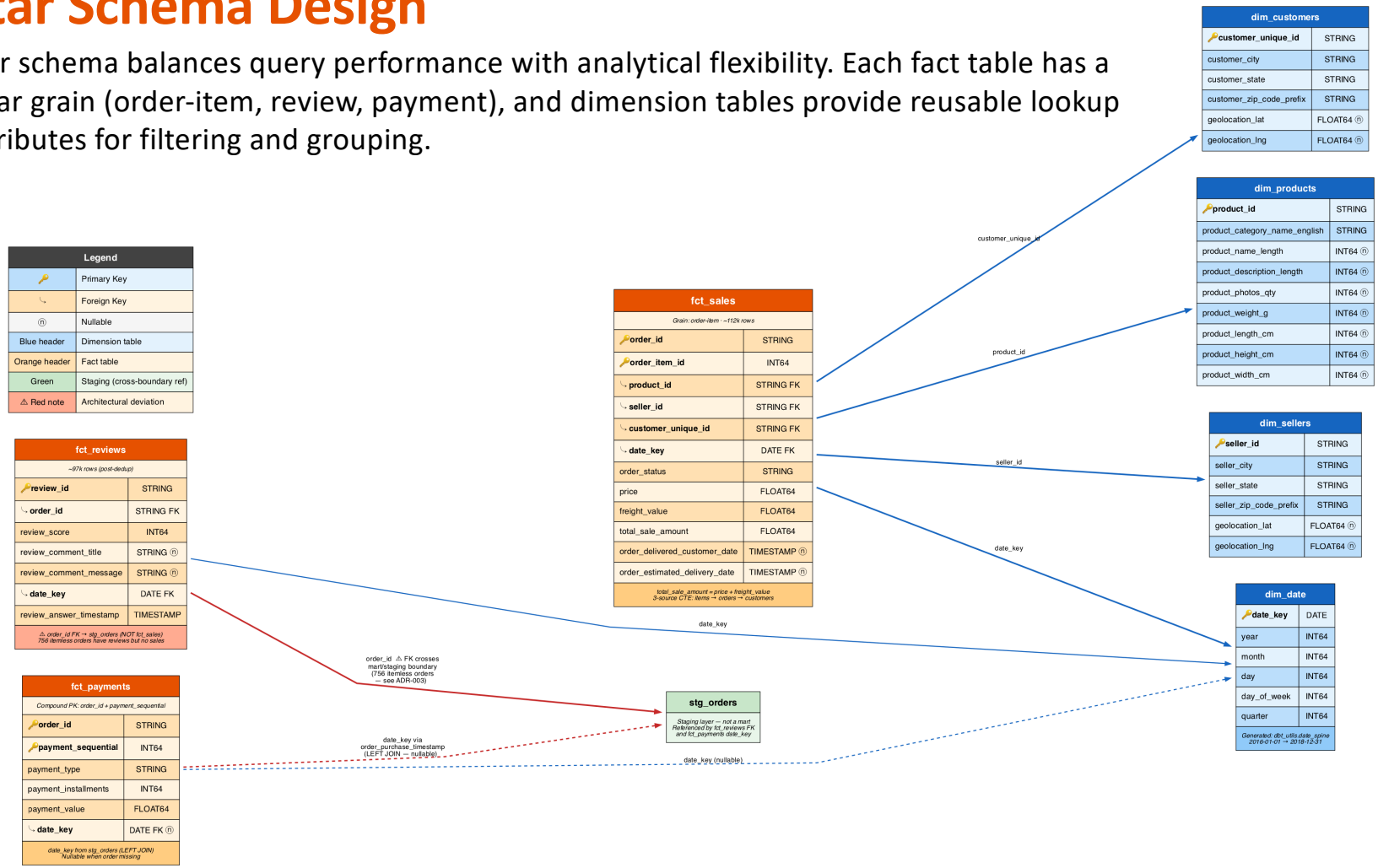
Generated: dbt_utils.date_spine (2016-01-01 to 2018-12-31)

No raw data source — see ADR-001

Why star schema? Single fact-to-dim joins for analytics • Dims evolve independently • Clear grain per fact table prevents 'universal fact table' anti-pattern • Staging layer (views) fixes all data defects once — every downstream consumer inherits clean data

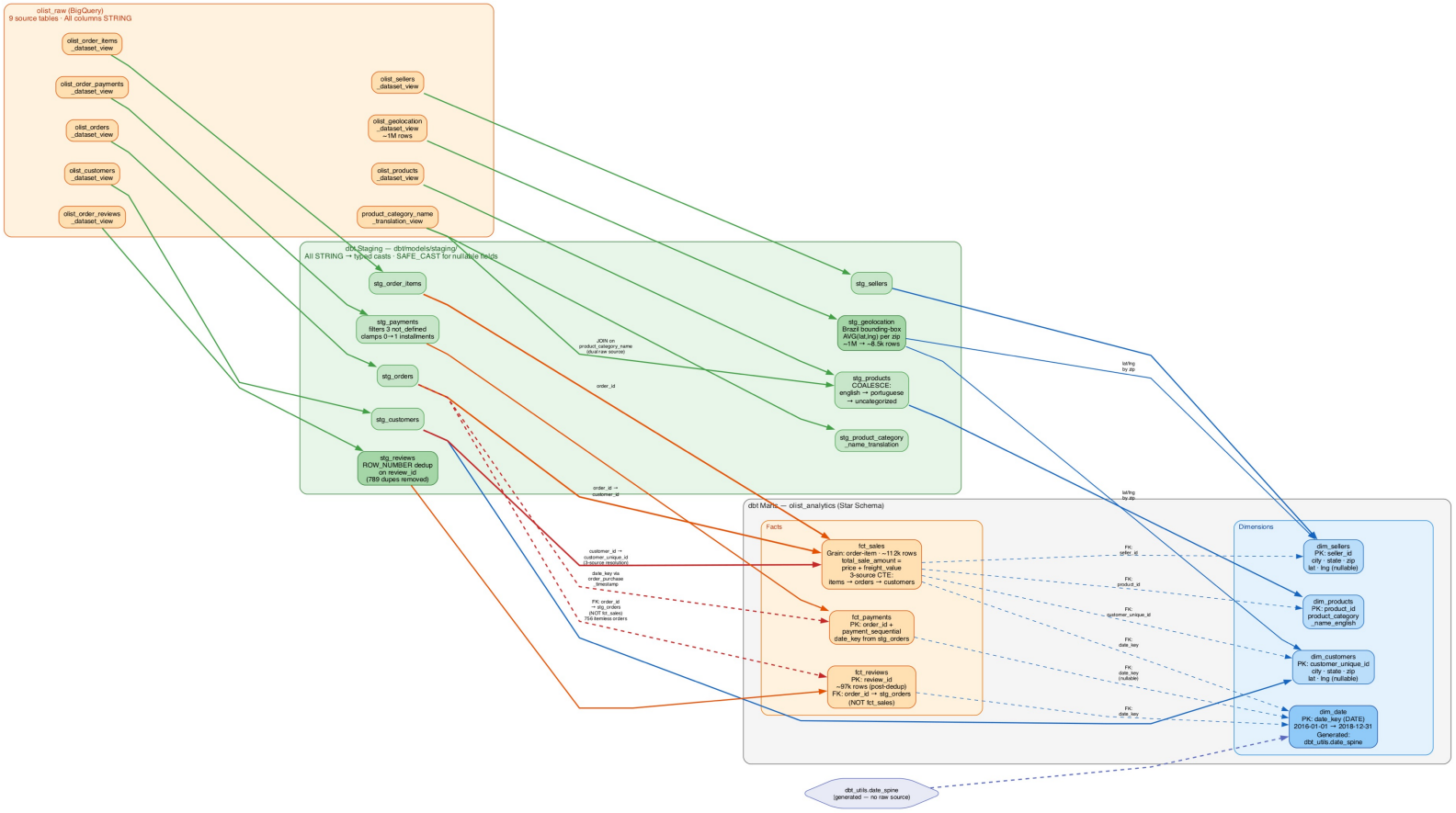
Star Schema Design

Star schema balances query performance with analytical flexibility. Each fact table has a clear grain (order-item, review, payment), and dimension tables provide reusable lookup attributes for filtering and grouping.



Data Lineage

In data engineering, this relationship is often described as a transformation hierarchy (similar to the Medallion Architecture of Bronze, Silver, and Gold). Each layer progressively refines the data from its messy original state into a polished, business-ready format.



Data Quality Framework

76 tests, 2 mechanisms, 0 failures — enforced at staging layer

Generic Tests (schema.yml)

not_null, unique — PK/FK enforcement
accepted_values — order_status, payment_type
expect_column_values_to_be_between — numeric ranges
expect_table_row_count_to_be_between — fan-out guards
unique_combination_of_columns — compound PKs
Temporal pair tests — timestamp ordering
relationships — FK referential integrity

Singular Tests (SQL files)

assert_boleto_single_installment.sql
Boleto payments must have exactly 1 instalment

assert_payment_reconciliation.sql
SUM(payment_value) ≈ SUM(total_sale_amount) per order

assert_date_key_range.sql
All fact table date_keys within 2016-01-01 to 2018-12-31

Staging Layer Data Defect Corrections

Model	Fix	Impact
stg_reviews	ROW_NUMBER dedup on review_id	789 duplicates removed
stg_geolocation	Brazil bounding-box filter + AVG(lat,lng) per zip	~1M → ~19k rows
stg_payments	Filter payment_type='not_defined'; clamp installments 0→1	3 + 2 rows fixed
stg_products	Rename 'lenght'→'length'; COALESCE category	610 products labelled 'uncategorized'

Architecture Decisions & Lessons Learned

ADR	Decision	Key Rationale
ADR-001	date_key as DATE (not INTEGER)	dbt_utils.date_spine natively produces DATE; INTEGER adds unnecessary casting everywhere
ADR-002	Dataset names: olist_raw / olist_analytics	'raw' is a BigQuery SQL reserved word; requires backtick-quoting
ADR-003	fct_reviews.order_id → stg_orders (not fct_sales)	756 itemless orders have reviews but no fct_sales rows; FK would orphan them
ADR-004	Meltano extractor: tap-csv	Streaming file reads; community-maintained; simpler config than tap-spreadsheets-anywhere

Non-Obvious Implementation Patterns

3-source CTE for fct_sales

- items → orders → customers (direct join = zero matches)
- customer_id is in stg_orders; customer_unique_id in stg_customers

batch_job method for Meltano loader

- gRPC idle timeout on 1M-row geolocation file
- Write API streams go idle during sequential file processing

@multi_asset(specs=...) for Dagster producer

- @asset(deps=...) made meltano_ingest downstream of raw tables
- @multi_asset declares it as the PRODUCER — correct topology

metaplane/dbt-expectations constraint

- Fork v0.10.8 has no 'mostly' parameter on any macro
- 4 proportion-based tests documented but unenforceable

AI Multi-Agent Architecture

Human Orchestrator + 5 Specialised AI Agents — contract-driven isolation

Agent	Role	Key Deliverables	Worktree
Agent 1a–1d	Data Engineer (4 phases)	Meltano config, 9 staging models, 7 mart models, 76 dbt tests	worktrees/agent-1
Agent 2	Platform Engineer	Dagster orchestration, schedule, launch scripts, .env integration	worktrees/agent-2
Agent 3	Analytics Engineer	4 Jupyter notebooks, 6 Parquet exports, utils.py, analytical methodology	worktrees/agent-3
Agent 4	Dashboard Engineer	Streamlit 5-page dashboard, 4 global filters, Lorenz/Gini charts	worktrees/agent-4
Agent 5	Data Scientist	Executive brief, slide deck, speaker notes, generate_slides.py	worktrees/agent-5
Orchestrator	Human + Claude (Opus)	Gate management, merge conflicts, diagrams, technical report, progress	main branch

Architecture Principles

Isolation: Each agent in a separate git worktree

Dependency gates: GATE-1→4 enforce execution order

Contract surfaces: CLAUDE.md (spec), sources.yml (Meltano↔dbt), Parquet schemas (notebooks↔dashboard)

42 changelog entries tracking all deviations

Agents communicate through shared specifications, not directly — the orchestrator manages gates and resolves merge conflicts across worktrees